

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل سوم: Data warehousing and online analytical processing

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 3.1 Data warehouse
 - 3.1.1 Data warehouse: what and why?
 - 3.1.2 Architecture of data warehouses: enterprise data warehouses and datamarts
 - 3.1.3 Data lakes
- 3.2 Data warehouse modeling: schema and measures
 - 3.2.1 Data cube: a multidimensional data model
 - 3.2.2 Schemas for multidimensional data models: stars, snowflakes, and factconstellations
 - 3.2.3 Concept hierarchies
 - 3.2.4 Measures: categorization and computation
- 3.3 OLAP operations
 - 3.3.1 Typical OLAP operations
 - 3.3.2 Indexing OLAP data: bitmap index and join index
 - 3.3.3 Storage implementation: column-based databases
- 3.4 Data cube computation
 - 3.4.1 Terminology of data cube computation
 - 3.4.2 Data cube materialization: ideas
 - 3.4.3 OLAP server architectures: ROLAP vs. MOLAP vs. HOLAP
 - 3.4.4 General strategies for data cube computation
- 3.5 Data cube computation methods
 - 3.5.1 Multiway array aggregation for full cube computation
 - 3.5.2 BUC: computing iceberg cubes from the apex cuboid downward
 - 3.5.3 Precomputing shell fragments for fast high-dimensional OLAP
 - 3.5.4 Efficient processing of OLAP queries using cuboids

Section 3.1

3.1.3. Data lakes

دریاچه داده (Data Lake)

دریاچه داده (Data Lake) یک مخزن متمرکز است که امکان ذخیره حجم عظیمی از داده‌های خام، بدون ساختار یا ساختاریافته را در مقیاس وسیع فراهم می‌کند. برخلاف پایگاه‌های داده سنتی یا انبارهای داده، دریاچه‌های داده انواع و فرمت‌های مختلف داده را در خود جای می‌دهند و راه‌حلی منعطف و مقیاس‌پذیر برای ذخیره و پردازش کارآمد داده‌ها ارائه می‌دهند. دریاچه‌های

داده اغلب در محیط‌های داده‌های بزرگ (big data) و تحلیل استفاده می‌شوند و به سازمان‌ها امکان می‌دهند حجم عظیمی از داده‌ها را به طور مؤثر ذخیره و تحلیل کنند.

```
In [10]: import pandas as pd
from sklearn.datasets import load_iris
import os

# مرحله اول: نوشتن داده‌ها
# استفاده می‌کنیم تا به داده‌های آماده دسترسی داشته باشیم (iris مثل) از یک دیتاست داخلی
# این دیتاست شامل ویژگی‌های گل‌های زنبق است
iris = load_iris()

# تبدیل می‌کنیم DataFrame داده‌ها و نام ویژگی‌ها را به یک
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# آن را در یک فایل موقت محلی ذخیره می‌کنیم، حالا به جای ذخیره در
# این فایل موقت، "دیتا لیک" ما را شبیه‌سازی می‌کند
file_name = 'iris_data.csv'
df.to_csv(file_name, index=False)
print(f"با موفقیت به عنوان دیتا لیک محلی ایجاد شد '{file_name}' فایل")

# مرحله دوم: خواندن داده‌ها
# حالا داده‌ها را از فایل موقتی که ایجاد کردیم، می‌خوانیم
df_from_lake = pd.read_csv(file_name)

# در نهایت، داده‌های خوانده‌شده از دیتا لیک را چاپ می‌کنیم
print("\n:داده‌های خوانده‌شده از دیتا لیک محلی")
print(df_from_lake.head())

# فایل موقت را حذف می‌کنیم تا سیستم را تمیز نگه داریم (اختیاری)
os.remove(file_name)
```

با موفقیت به عنوان دیتا لیک محلی ایجاد شد 'iris_data.csv' فایل

داده‌های خوانده‌شده از دیتا لیک محلی:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

توضیحات مثال کد پایتون

این مثال یک نمایش ساده‌شده از کار با مفهوم **دریاچه داده** ارائه می‌دهد. به جای استفاده از یک سرویس ابری مانند **Amazon S3**، ما یک دریاچه داده محلی را با استفاده از یک فایل **CSV** بر روی کامپیوترمان شبیه‌سازی می‌کنیم.

کد ابتدا یک مجموعه داده آماده برای استفاده از کتابخانه **Scikit-learn**، به ویژه مجموعه داده معروف **Iris**، را بارگذاری می‌کند.

سپس نحوه ذخیره یک **DataFrame** پانداس به عنوان یک فایل **CSV** در دریاچه داده محلی خود و متعاقباً بازیابی و نمایش داده‌ها از آن فایل را نشان می‌دهد.

این فرآیند، هرچند ساده‌شده است، عملیات اساسی یک دریاچه داده واقعی را منعکس می‌کند و نشان می‌دهد که چگونه داده‌ها را می‌توان در یک مخزن متمرکز نوشت و از آن خواند. این رویکرد برای یادگیری و تست، بدون نیاز به پیچیدگی‌های پیچیده ابری، عالی است.

Section 3.2

3.2.1. Data cube: a multidimensional data model

مکعب داده (Data Cube)

مکعب داده (Data Cube) یک مدل داده چندبعدی است که برای نمایش و تحلیل داده‌ها در ابعاد مختلف استفاده می‌شود. این مدل ساختار جدول سنتی و دوبعدی پایگاه‌های داده رابطه‌ای را گسترش می‌دهد تا ابعاد اضافی را در خود جای دهد و یک نمای جامع‌تر از داده‌ها ارائه کند. در یک مکعب داده، هر خانه (cell) حاوی یک مقدار یا معیار است و می‌توان مکعب را برای کاوش و

تحلیل داده‌ها از دیدگاه‌های مختلف، برش (slice)، تکه‌تکه (dice) یا جمع‌آوری (roll up) کرد. این نمایش چندبعدی به‌ویژه برای کاربردهای داده‌کاوی و هوش تجاری (business intelligence) مفید است.

مثال مکعب داده در پایتون

اگرچه کتابخانه‌ای به نام "data cube" به طور خاص در پایتون وجود ندارد، اما می‌توانید از کتابخانه‌هایی مانند **pandas** و **matplotlib** برای ایجاد بصری‌سازی‌هایی که داده‌های چندبعدی را نمایش می‌دهند، استفاده کنید. در این مثال، من نحوه ایجاد یک نمودار پراکندگی سه‌بعدی (3D scatter plot) ساده را نشان می‌دهم که می‌تواند به عنوان یک نمایش ساده‌شده از یک مکعب داده سه‌بعدی در نظر گرفته شود.

```
In [11]: import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# این قسمت اول کده. سه تا کتابخونه اصلی رو وارد می‌کنیم
# 'pandas' برای کار با داده‌ها تو قالب جدول
# 'matplotlib.pyplot' برای رسم نمودارها
# 'Axes3D' برای رسم نمودارهای سه‌بعدی هست که ابزاری از 'matplotlib'

# 'X', 'Y' و 'Z' دارن (Dimension) این دیکشنری حاوی داده‌های نمونه ماست که سه ویژگی یا بعد
# در نظر بگیرید (Data Cube) این داده‌ها رو به عنوان یک مکعب داده.
data = {
    'X': [1, 2, 3, 4, 5],
    'Y': [5, 4, 3, 2, 1],
    'Z': [10, 8, 6, 4, 2]
}

# جدید می‌سازیم 'DataFrame' با استفاده از داده‌های بالا، به
df = pd.DataFrame(data)

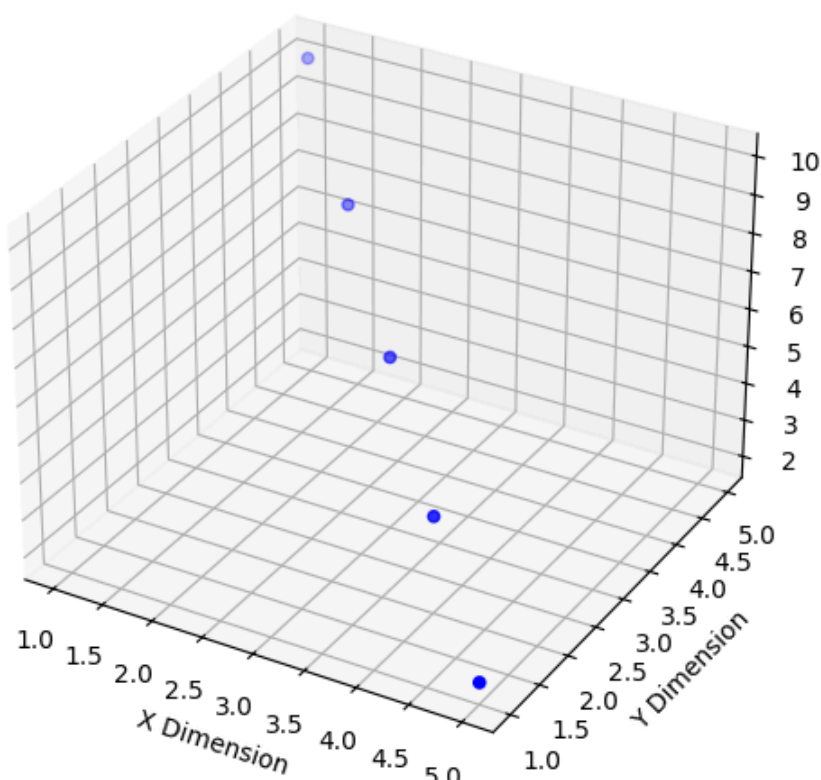
# این بخش، یک **نمودار پراکندگی سه‌بعدی** رو ایجاد می‌کنه.
# این نمودار برای بصری‌سازی داده‌هایی با سه ویژگی یا بعد عالیه.
# محوره‌های سه‌بعدی رو ایجاد می‌کنه 'ax' فضای رسم و 'fig' رسم.
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

# تابع اصلی برای رسم نقاط تو فضای سه‌بعدیه 'ax.scatter()'
# 'X', 'Y' و 'Z' مشخص می‌کنن 'marker' رنگ نقاط و 'c'
ax.scatter(df['X'], df['Y'], df['Z'], c='blue', marker='o')

# این خطوط برای زیباتر شدن و قابل فهم‌تر شدن نمودار هستن.
# با توجه به درخواست شما، عنوان و برچسب محورها به انگلیسی نوشته شدن.
ax.set_xlabel('X Dimension')
ax.set_ylabel('Y Dimension')
ax.set_zlabel('Z Dimension')
ax.set_title('3D Scatter Plot: Simulated Data Cube')

# نمودار سه‌بعدی رو نمایش می‌ده 'plt.show()'
plt.show()
```

3D Scatter Plot: Simulated Data Cube



توضیحات مثال کد پایتون

در این مثال، دیتافریم `df` نمایانگر یک مجموعه داده ساده سه‌بعدی با ابعاد `X`، `Y` و `Z` است. کد با استفاده از کتابخانه `matplotlib` یک نمودار پراکندگی سه‌بعدی ایجاد می‌کند تا نقاط داده را در فضای سه‌بعدی بصری‌سازی کند. اگرچه این مثال به‌طور کامل پیچیدگی‌های یک مکعب داده واقعی را به تصویر نمی‌کشد، اما مفهوم نمایش داده‌ها در ابعاد چندگانه را نشان می‌دهد. موارد استفاده واقعی از مکعب‌های داده شامل مجموعه‌داده‌های گسترده‌تر و ابزارهای پیشرفته‌ای مانند سیستم‌های OLAP (پردازش تحلیلی آنلاین) است که به کاربران اجازه می‌دهد داده‌های چندبعدی را به صورت تعاملی تحلیل کنند.

3.2.2. Schemas for multidimensional data models: stars, snowflakes, and fact constellations

شمای چندبعدی در مدل‌سازی داده

در مدل‌سازی داده‌های چندبعدی، شماها (schemas) ساختارهایی هستند که نحوه سازماندهی و ارتباط داده‌ها را در یک پایگاه داده تعریف می‌کنند. سه نوع شمای رایج برای مدل‌های داده چندبعدی شامل ستاره‌ای (star)، برف‌دانه (snowflake) و صورت فلکی (fact constellation) می‌باشند.

شمای ستاره‌ای (Star Schema): ✨

در شمای ستاره‌ای، یک جدول مرکزی به نام جدول واقعیت (fact table) وجود دارد که حاوی معیارهای اصلی است. این جدول توسط جداول ابعاد (dimension tables) احاطه شده است که هر یک جنبه یا دیدگاه متفاوتی از داده‌ها را نشان می‌دهند. این ساختار از نظر بصری شبیه به یک ستاره است.

مثال در پایتون:

اگرچه مفهوم شمای ستاره‌ای بیشتر به طراحی پایگاه داده مربوط است، اما می‌توانید یک شمای ستاره‌ای ساده را با استفاده از دیتافریم‌های `pandas` شبیه‌سازی کنید. یک مجموعه داده فروش را با یک جدول واقعیت مرکزی حاوی معیارهای فروش و جداول ابعاد پیرامونی برای محصولات، مشتریان و زمان در نظر بگیرید.

شمای برف‌دانه (Snowflake Schema): ❄️

شمای برف‌دانه، شمای ستاره‌ای را با نرمال‌سازی جداول ابعاد گسترش می‌دهد و آن‌ها را به زیر-ابعاد یا سطوح تقسیم می‌کند. این امر منجر به ساختاری می‌شود که از نظر بصری شبیه به یک برف‌دانه است.

مثال در پایتون:

مجدداً، با استفاده از `pandas`، می‌توانید با نرمال‌سازی جداول ابعاد از مثال شمای ستاره‌ای، یک شمای برف‌دانه را شبیه‌سازی کنید. این کار ممکن است شامل تقسیم بعد "محصولات" به زیر-ابعادی مانند "دسته‌بندی محصول" و "زیر-دسته‌بندی محصول" باشد.

شمای صورت فلکی (Fact Constellation): 🌟

شمای صورت فلکی شامل چندین جدول واقعیت است که جداول ابعاد مشترکی دارند. هر جدول واقعیت معیارهای متفاوتی را نشان می‌دهد و ابعاد مشترک امکان تحلیل متقابل بین جداول واقعیت را فراهم می‌کنند.

مثال در پایتون:

مثال شمای ستاره‌ای را با افزودن یک جدول واقعیت دیگر، مانند جدول واقعیت "هزینه‌ها"، گسترش دهید. هر دو جدول واقعیت "فروش" و "هزینه‌ها" ابعاد مشترکی مانند "زمان" و "مشتری" را به اشتراک می‌گذارند.

```
In [12]: import pandas as pd
import numpy as np

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالی‌ه 'pandas' این قسمت اول کده. ما کتابخونه

# رو ایجاد می‌کنه 'Sales' به نام (**Fact Table) این بخش، **جدول واقعیت
# جدول واقعیت، رویدادها یا تراکنش‌های اصلی کسب و کار رو شامل می‌شه
# رو تو خودش نگه می‌داره (Dimension) و کلیدهایی برای ارتباط با جدول‌های ابعاد ('SalesAmount' مثل) این جدول معمولاً اعداد قابل اندازه‌گیری
sales_data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-01-03'],
    'ProductID': [1, 2, 1],
    'CustomerID': [101, 102, 103],
    'SalesAmount': [500, 300, 700]
}

sales_df = pd.DataFrame(sales_data)

# رو ایجاد می‌کنه (**Dimension Tables) این بخش، **جدول‌های ابعاد
# جدول‌های ابعاد، جزئیات توصیفی رو برای جدول واقعیت فراهم می‌کنن
# داره 'ProductID' رو برای هر 'Category' و 'ProductName' اطلاعاتی مثل 'Products' مثلاً جدول
# معروفه (**Star Schema) این ساختار که یک جدول واقعیت مرکزی و چند جدول ابعاد اطرافش داره، به **طرح‌واره ستاره‌ای
products_data = {
    'ProductID': [1, 2],
    'ProductName': ['ProductA', 'ProductB'],
    'Category': ['Electronics', 'Clothing']
}

customers_data = {
    'CustomerID': [101, 102, 103],
    'CustomerName': ['Alice', 'Bob', 'Charlie'],
    'City': ['New York', 'San Francisco', 'Los Angeles']
}

products_df = pd.DataFrame(products_data)
customers_df = pd.DataFrame(customers_data)

# این بخش، جدول‌ها رو چاپ می‌کنه تا ساختار طرح‌واره ستاره‌ای رو نشون بده
print("Sales: جدول واقعیت")
print(sales_df)
print("Products: جدول ابعاد")
print(products_df)
print("Customers: جدول ابعاد")
print(customers_df)
```

Sales: جدول واقعیت

	Date	ProductID	CustomerID	SalesAmount
0	2022-01-01	1	101	500
1	2022-01-02	2	102	300
2	2022-01-03	1	103	700

Products: جدول ابعاد

	ProductID	ProductName	Category
0	1	ProductA	Electronics
1	2	ProductB	Clothing

Customers: جدول ابعاد

	CustomerID	CustomerName	City
0	101	Alice	New York
1	102	Bob	San Francisco
2	103	Charlie	Los Angeles



توضیحات مثال کد پایتون

مفاهیم شمای ستاره‌ای (star)، برف‌دانه (snowflake) و صورت فلکی (fact constellation)، با تأکید بر تفاوت‌های آن‌ها در ساختار و موارد استفاده نشان داده شد.

3.2.3. Concept hierarchies



سلسله مراتب مفاهیم

سلسله مراتب مفاهیم (Concept hierarchies) به سازماندهی داده‌ها در سطوح انتزاعی یا گرانولاریته اشاره دارد که یک سلسله مراتب از مفاهیم را تشکیل می‌دهد. در زمینه داده‌کاوی، سلسله مراتب مفاهیم اغلب بر روی ابعاد (dimensions) در مدل‌های

داده چندبعدی اعمال می‌شود. هر سطح از سلسله مراتب، درجه متفاوتی از خلاصه‌سازی یا جزئیات را نشان می‌دهد و به کاربران امکان می‌دهد داده‌ها را در سطوح مختلفی از جزئیات کاوش کنند.

مثال سلسله مراتب مفاهیم در پایتون

بیایید یک مثال ساده را با استفاده از یک دیتافریم پانداس در نظر بگیریم که داده‌های فروش را با یک بعد زمانی نشان می‌دهد و دارای سلسله مراتب مفهومی سال، فصل و ماه است.

```
In [13]: import pandas as pd
import numpy as np

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه

# حاوی داده‌های فروش نمونه ماست (dictionary) این دیکشنری
# یک ویژگی زمانیۀ که می‌خوایم روی اون سلسله مراتب مفهومی رو ایجاد کنیم 'Date'
data = {
    'Date': ['2022-01-01', '2022-04-15', '2022-06-30', '2022-11-20'],
    'ProductID': [1, 2, 1, 3],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو ایجاد می‌کنه ** (Concept Hierarchy) این بخش، ** سلسله مراتب مفهومی
# سلسله مراتب مفهومی، راهی برای سازماندهی و خلاصه‌سازی داده‌ها از سطح پایین‌تر به سطح بالاتر هست
# برای داده‌های زمانی، ما می‌تونیم از روز به ماه، فصل، و سال بریم
# سال رو از تاریخ استخراج می‌کنه 'dt.year'
sales_df['Year'] = sales_df['Date'].dt.year
# فصل رو از تاریخ استخراج می‌کنه 'dt.to_period("Q")'
sales_df['Quarter'] = sales_df['Date'].dt.to_period("Q")
# ماه رو از تاریخ استخراج می‌کنه 'dt.to_period("M")'
sales_df['Month'] = sales_df['Date'].dt.to_period("M")

# جدید رو چاپ می‌کنیم تا ببینیم سلسله مراتب زمانی چطور اضافه شده 'DataFrame'، در نهایت
print("داده‌های فروش با سلسله مراتب مفهومی زمانی:")
print(sales_df[['Date', 'Year', 'Quarter', 'Month', 'ProductID', 'SalesAmount']])
```

داده‌های فروش با سلسله مراتب مفهومی زمانی:

	Date	Year	Quarter	Month	ProductID	SalesAmount
0	2022-01-01	2022	2022Q1	2022-01	1	500
1	2022-04-15	2022	2022Q2	2022-04	2	300
2	2022-06-30	2022	2022Q2	2022-06	1	700
3	2022-11-20	2022	2022Q4	2022-11	3	450

توضیحات مثال کد پایتون

در این مثال، ستون تاریخ با سطوح اضافی سلسله مراتبی مبتنی بر زمان تقویت شده است: سال، فصل و ماه. این امر به کاربران امکان می‌دهد داده‌های فروش را در سطوح مختلفی از گرانولاریته زمانی تحلیل کنند.

سلسله مراتب مفاهیم با فراهم‌کردن روشی ساختارمند برای حرکت در داده‌ها در سطوح مختلف انتزاع، تحلیل داده را بهبود می‌بخشند. کاربران می‌توانند داده‌ها را بر اساس نیاز تحلیلی خود تجمیع کنند یا به جزئیات بیشتری فرو بروند. این مثال نحوه ایجاد یک سلسله مراتب مفهومی ساده برای بعد زمانی در یک مجموعه داده فروش را نشان می‌دهد، اما سلسله مراتب مفاهیم می‌توانند در سناریوهای واقعی به ابعاد مختلفی مانند جغرافیا، دسته‌های محصول یا سلسله مراتب سازمانی نیز اعمال شوند.

3.2.4. Measures: categorization and computation

معیارها در داده‌کاوی

در زمینه داده‌کاوی، معیارها (measures) به مقادیر کمی یا شاخص‌هایی اطلاق می‌شوند که مورد علاقه بوده و اغلب در فرآیندهای تحلیلی استفاده می‌شوند. معیارها بسته به ماهیت خود قابل دسته‌بندی هستند، مانند جمع‌پذیر یا غیرجمع‌پذیر، و

ممکن است مبتنی بر محاسباتی باشند که بینش‌های معناداری از داده‌ها استخراج می‌کنند.

دسته‌بندی معیارها¹²³⁴

معیارهای جمع‌پذیر: معیارهای جمع‌پذیر، معیارهایی هستند که می‌توان آن‌ها را در ابعاد مختلف تجمیع یا جمع کرد. از جمله مثال‌های آن می‌توان به کل فروش، سود کل یا کل تعداد فروخته‌شده اشاره کرد.

معیارهای غیرجمع‌پذیر: معیارهای غیرجمع‌پذیر را نمی‌توان به سادگی در تمام ابعاد تجمیع کرد. از جمله مثال‌های آن می‌توان به میانگین حاشیه سود، درصد رشد یا نسبت‌ها اشاره کرد.

محاسبه معیارها: محاسبه به معنای استخراج معیارهای جدید یا تبدیل معیارهای موجود برای کسب بینش عمیق‌تر از داده‌هاست. روش‌های رایج محاسبه شامل یافتن درصدها، نرخ رشد، میانگین‌ها یا هر عملیات ریاضی مرتبط دیگری است.

مثالی از معیارها در پایتون

بیاپید یک مجموعه داده فروش را در نظر بگیریم و هم معیارهای جمع‌پذیر و هم معیارهای غیرجمع‌پذیر را محاسبه کنیم.

In [14]:

```
import pandas as pd
import numpy as np

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# این داده‌ها رو برای نمایش تفاوت بین سنجه‌های جمع‌پذیر و غیرجمع‌پذیر استفاده می‌کنیم
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-01-03', '2022-01-04'],
    'ProductID': [1, 2, 1, 3],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو محاسبه می‌کنه (** Additive Measure) این بخش، **سنجه جمع‌پذیر
# سنجۀ جمع‌پذیر، سنجۀ‌ای هست که می‌تونیم اون رو روی ابعاد مختلف (مثل زمان یا محصول) جمع کنیم و نتیجه معنادار باشه
# یا 'کل فروش' یک سنجۀ جمع‌پذیره، چون جمع فروش روزهای مختلف، کل فروش رو می‌ده 'Total Sales' مثلاً
total_sales = sales_df['SalesAmount'].sum()

# رو محاسبه می‌کنه (** Non-additive Measure) این بخش، **سنجه غیرجمع‌پذیر
# سنجۀ غیرجمع‌پذیر، سنجۀ‌ای هست که جمع کردنش روی ابعاد مختلف، معنی نداره
# یا 'میانگین فروش' یک سنجۀ غیرجمع‌پذیره، چون میانگین میانگین‌های روزانه، میانگین کل رو نمی‌ده 'Average Sales'
average_sales = sales_df['SalesAmount'].mean()

# در نهایت، نتایج محاسبه‌شده رو چاپ می‌کنیم تا تفاوتشون رو ببینیم
print(" = سنجه جمع‌پذیر: کل فروش", total_sales)
print(" = سنجه غیرجمع‌پذیر: میانگین فروش", average_sales)
```

سنجه جمع‌پذیر: کل فروش = 1950

سنجه غیرجمع‌پذیر: میانگین فروش = 487.5

توضیحات مثال کد پایتون

در این مثال، ستون SalesAmount نشان‌دهنده یک معیار است و ما از داده‌های فروش، یک معیار جمع‌پذیر (کل فروش) و یک معیار غیرجمع‌پذیر (میانگین فروش) را محاسبه می‌کنیم.

درک و دسته‌بندی معیارها برای تحلیل مؤثر داده‌ها ضروری است. معیارهای مختلف ممکن است بر اساس اهداف تحلیلی، روش‌های محاسباتی متفاوتی نیاز داشته باشند. این مثال نحوه محاسبه معیارهای جمع‌پذیر و غیرجمع‌پذیر در یک مجموعه داده

Section 3.3

3.3.1. Typical OLAP operations

پردازش تحلیلی آنی (OLAP)

OLAP یا پردازش تحلیلی آنی (Online Analytical Processing)، دسته‌ای از ابزارها و فناوری‌ها است که به کاربران امکان می‌دهد به صورت تعاملی داده‌های چندبعدی را برای پشتیبانی از تصمیم‌گیری و هوش تجاری تحلیل کنند. عملیات OLAP، عملیات پایه‌ای هستند که کاربران بر روی مکعب‌های داده چندبعدی انجام می‌دهند تا بینش‌های ارزشمندی استخراج کنند.

عملیات متداول OLAP

عملیات فشردن (Roll-up / Drill-Up): داده‌ها را در امتداد یک بعد تجمیع می‌کند و از سطح پایین‌تر گرانولاریته به سطح بالاتر می‌رود. مثلاً، فشردن داده‌های فروش ماهانه به فصلی.

عملیات جزئی‌سازی (Drill-down / Roll-Down): داده‌های تجمیع‌شده را در امتداد یک بعد باز می‌کند و از سطح بالاتر به سطح پایین‌تر گرانولاریته می‌رود. مثلاً، جزئی‌سازی داده‌های فروش سالانه به ماهانه.

عملیات برش (Slice): با انتخاب یک سطح از یک بعد، یک برش دو بعدی از مکعب ایجاد می‌کند. مثلاً، انتخاب داده‌های فروش یک فصل خاص و یک دسته محصول.

عملیات تاس‌زدن (Dice): با مشخص کردن مقادیر در چند بعد، یک زیرمکعب انتخاب می‌کند. مثلاً، انتخاب داده‌های فروش یک منطقه، فصل و دسته محصول خاص.

عملیات چرخش (Pivot / Rotate): مکعب داده را می‌چرخاند تا داده‌ها از دیدگاه دیگری نمایش داده شوند. مثلاً، چرخش برای مشاهده فروش محصولات در طول فصل‌های مختلف.

مثالی از عملیات OLAP در پایتون

اگرچه کتابخانه‌های تخصصی OLAP در پایتون وجود ندارند، اما می‌توان از کتابخانه pandas برای شبیه‌سازی عملیات OLAP بر روی یک مجموعه داده چندبعدی استفاده کرد. بیایید یک مجموعه داده ساده فروش را در نظر بگیریم و عملیات‌های برش (Slice) و تاس‌زدن (Dice) را انجام دهیم.

```
In [15]: import pandas as pd
import numpy as np
```

رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه #


```
# حاوی داده‌های فروش نمونه ماست (dictionary) این دیکشنری
# رو داره 'Date' و 'Region' در نظر بگیرید که ابعاد مختلفی مثل (Data Cube) این داده‌ها رو به عنوان یک مکعب داده
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# اصلی رو چاپ می‌کنیم تا ببینیم داده‌ها چه شکلی هستن 'DataFrame'
print("داده‌های فروش اصلی:")
print(sales_df)

# رو انجام می‌ده ** (Slice) این بخش، عملیات ** برش
# برش، انتخاب داده‌ها بر اساس یک بعد خاصه
# اینجا، ما فقط داده‌های مربوط به ماه ژانویه (ماه 1) رو انتخاب می‌کنیم
january_sales = sales_df[sales_df['Date'].dt.month == 1]

# رو انجام می‌ده ** (Dice) این بخش، عملیات ** مکعب داده
# مکعب داده، انتخاب داده‌ها بر اساس ترکیبی از چند بعد
# باشن و هم مربوط به ماه ژانویه 'North' اینجا، ما داده‌هایی رو انتخاب می‌کنیم که هم مربوط به منطقه
north_january_sales = sales_df[(sales_df['Region'] == 'North') & (sales_df['Date'].dt.month == 1)]

# در نهایت، نتایج هر دو عملیات رو چاپ می‌کنیم
print("\nبرش (فروش ژانویه) OLAP عملیات:")
print(january_sales)

print("\nNorth فروش ژانویه و منطقه مکعب داده OLAP عملیات:")
print(north_january_sales)
```

داده‌های فروش اصلی:

	Date	ProductID	Region	SalesAmount
0	2022-01-01	1	North	500
1	2022-01-02	2	South	300
2	2022-02-01	1	North	700
3	2022-02-02	2	South	450

برش (فروش ژانویه) OLAP عملیات:

	Date	ProductID	Region	SalesAmount
0	2022-01-01	1	North	500
1	2022-01-02	2	South	300

North فروش ژانویه و منطقه مکعب داده OLAP عملیات:

	Date	ProductID	Region	SalesAmount
0	2022-01-01	1	North	500



توضیحات مثال کد پایتون

در این مثال، یک عملیات برش (Slice) با انتخاب داده‌های فروش ماه ژانویه و یک عملیات تاس‌زدن (Dice) با انتخاب داده‌های فروش منطقه شمال در ماه ژانویه انجام شده است.

اگرچه عملیات OLAP معمولاً با استفاده از ابزارها و پایگاه‌های داده تخصصی انجام می‌شوند، اما می‌توان از **pandas** برای شبیه‌سازی این عملیات روی مجموعه داده‌های کوچکتر استفاده کرد.

3.3.2. Indexing OLAP data: bitmap index and join index



نقش ایندکس‌گذاری در OLAP

در پایگاه‌های داده OLAP، **ایندکس‌گذاری** نقشی حیاتی در بهینه‌سازی عملکرد پرس‌وجو و تسریع بازیابی داده‌ها ایفا می‌کند. دو نوع متداول از ایندکس‌ها که در OLAP استفاده می‌شوند، **ایندکس بیت‌مپ (Bitmap Index)** و **ایندکس جویین (Join Index)** هستند.



انواع ایندکس‌ها در OLAP

ایندکس بیت‌مپ (Bitmap Index): در این تکنیک ایندکس‌گذاری، برای هر مقدار منحصر‌به‌فرد در یک بعد، یک نقشه بیتی (bitmap) در نظر گرفته می‌شود. هر بیت در این نقشه مربوط به یک سطر در جدول واقعیت (fact table) است و نشان می‌دهد که آیا آن مقدار بعد در آن سطر وجود دارد یا خیر. ایندکس‌های بیت‌مپ از نظر فضای ذخیره‌سازی کارآمد هستند و به‌ویژه برای ویژگی‌های با **کاردینالیت‌ه پایین** (تعداد مقادیر مجزای کم) مناسب می‌باشند.

ایندکس جویین (Join Index): ایندکس جویین برای بهینه‌سازی عملیات اتصال (join) بین جداول واقعیت و ابعاد طراحی شده است. این ایندکس نتایج از پیش محاسبه‌شده‌ای از جویین‌های خاص را ذخیره می‌کند و نیاز به اجرای جویین‌های پرهزینه در زمان اجرای پرس‌وجو را کاهش می‌دهد. ایندکس جویین به‌ویژه در پرس‌وجوهای پیچیده که شامل چندین بعد می‌شوند، **بسیار مفید** است.

مثالی از ایندکس بیت‌مپ و ایندکس جویین در پایتون 🐍

برای مثالی ساده‌شده، از کتابخانه pandas استفاده می‌کنیم تا سناریویی را شبیه‌سازی کنیم که در آن یک **ایندکس بیت‌مپ** برای یک بعد ایجاد می‌شود و همچنین از یک **ایندکس جویین** برای بهینه‌سازی عملیات اتصال استفاده می‌شود.

```
In [16]: import pandas as pd
import numpy as np

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالی‌ه 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو شبیه‌سازی می‌کنه ** (Bitmap Index) این بخش، **/ایندکس بیت‌مپ
# ایندکس بیت‌مپ برای ستون‌هایی که تعداد مقادیر منحصر‌به‌فردشون کمه، خیلی کارآمده
# ایندکس برای هر مقدار یک بیت‌مپ (آرایه‌ای از 0 و 1) می‌سازه
# باشه، 1 و بقیه 0 می‌شن 'North' برابر 'Region' هر جایی که 'North' مثلاً برای
# این کار جستجو رو خیلی سریع‌تر می‌کنه
bitmap_index_region = {}
for region in sales_df['Region'].unique():
    bitmap_index_region[region] = sales_df['Region'] == region

# رو شبیه‌سازی می‌کنه ** (Join Index) این بخش، **/ایندکس پیوست
# رو از قبل ذخیره می‌کنه (Join) ایندکس پیوست، نتایج یک عملیات پیوست
# دو جدول داریم، دیگه نخوایم کل عملیات رو از اول انجام بدیم و سرعت جستجو و بازیابی داده‌ها بالا بره (Join) این کار باعث می‌شه وقتی نیاز به پیوست
# می‌کنیم (join) پیوست 'product_dimension' رو با یک جدول ابعاد 'sales_df' ما اینجا جدول
product_dimension = pd.DataFrame({
    'ProductID': [1, 2],
    'ProductName': ['ProductA', 'ProductB']
})

join_index_product = pd.merge(sales_df, product_dimension, on='ProductID', how='inner')

# در نهایت، ایندکس‌های شبیه‌سازی شده رو چاپ می‌کنیم
print("Region:")
print(bitmap_index_region)

print("\nProduct:")
print(join_index_product)
```

```
'Region':  
{'North': 0      True  
1      False  
2      True  
3      False  
Name: Region, dtype: bool, 'South': 0      False  
1      True  
2      False  
3      True  
Name: Region, dtype: bool}
```

```
'Product':  
      Date  ProductID Region  SalesAmount ProductName  
0 2022-01-01         1   North          500   ProductA  
1 2022-01-02         2   South          300   ProductB  
2 2022-02-01         1   North          700   ProductA  
3 2022-02-02         2   South          450   ProductB
```

توضیحات مثال کد پایتون

در این مثال، یک **Bitmap Index** برای بعد 'Region' شبیه‌سازی شده است، که در آن هر مقدار در دیکشنری نشان‌دهنده یک بیت‌مپ برای یک مقدار منحصر به فرد از منطقه است. همچنین یک **Join Index** برای بعد 'Product' با ادغام داده‌های فروش و جدول جداگانه محصولات شبیه‌سازی شده است.

اگرچه این شبیه‌سازی‌ها ساده هستند، اما پیاده‌سازی واقعی **Bitmap** و **Join Index** در پایگاه‌های داده تخصصی OLAP انجام می‌شود. این ایندکس‌ها عملکرد پرس‌وجوها را در سناریوهایی که شامل فیلتر کردن و اتصال داده‌ها در ابعاد مختلف می‌شوند، به‌طور چشمگیری بهبود می‌دهند.

3.3.3. Storage implementation: column-based databases

پایگاه‌های داده مبتنی بر ستون

پایگاه‌های داده مبتنی بر ستون نوعی از سیستم‌های مدیریت پایگاه داده (DBMS) هستند که داده‌ها را بر اساس ستون‌ها — نه ردیف‌ها — ذخیره و سازماندهی می‌کنند. در این نوع پایگاه داده، هر ستون به‌صورت جداگانه ذخیره می‌شود که امکان بازیابی داده و تحلیل کارآمدتری فراهم می‌کند، به‌ویژه هنگام کار با مجموعه داده‌های بزرگ. این ساختار ذخیره‌سازی برای کاربردهای داده‌کاوی و تحلیل داده بسیار مفید است، زیرا **عملکرد پرس‌وجو** را بهبود می‌بخشد و از تکنیک‌های **فشرده‌سازی داده** پشتیبانی می‌کند.

ویژگی‌های کلیدی پایگاه‌های داده مبتنی بر ستون

ذخیره‌سازی ستونی (Columnar Storage): داده‌ها به‌جای فرمت ردیفی، به صورت ستونی ذخیره می‌شوند. هر ستون به‌عنوان یک فایل یا واحد ذخیره‌سازی مجزا نگهداری می‌شود.

تکنیک‌های فشرده‌سازی (Compression Techniques): پایگاه‌های داده مبتنی بر ستون اغلب از روش‌های فشرده‌سازی استفاده می‌کنند که با استفاده از یکنواختی داده‌ها درون هر ستون، فضای ذخیره‌سازی را کاهش داده و **عملکرد پرس‌وجو** را بهبود می‌بخشند.

کارایی در پردازش تحلیلی (Analytical Processing Efficiency): این پایگاه‌های داده برای کارهای تحلیلی و داده‌کاوی بسیار مناسب هستند، به‌ویژه زمانی که عملیات‌های رایج شامل تجمیع و تحلیل روی زیرمجموعه‌ای از ستون‌ها باشند.

مثالی از ذخیره‌سازی مبتنی بر ستون در پایتون

اگرچه پایتون به‌خودی‌خود دارای فرمت ذخیره‌سازی مبتنی بر ستون نیست، اما می‌توان از کتابخانه‌هایی مانند Pandas برای شبیه‌سازی برخی از جنبه‌های ذخیره‌سازی مبتنی بر ستون استفاده کرد. بیایید یک مثال ساده را در نظر بگیریم:

In [17]: **import** pandas **as** pd

رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه

این دیکشنری حاوی داده‌های فروش نمونه ماست.

استفاده می‌کنیم (Column-based Storage) این داده‌ها رو برای شبیه‌سازی ذخیره‌سازی مبتنی بر ستون

```
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}
```

sales_df = pd.DataFrame(data)

تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون

تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم

sales_df['Date'] = pd.to_datetime(sales_df['Date'])

رو شبیه‌سازی می‌کنه ** (Column-based Storage) این بخش، **ذخیره‌سازی مبتنی بر ستون

(Amazon Redshift یا Apache Parquet مثلاً) در سیستم‌های واقعی مثل پایگاه‌های داده ستونی

داده‌ها به جای ذخیره کردن سطر به سطر، ستون به ستون ذخیره می‌شن

این کار باعث می‌شه عملیاتی مثل جمع‌کردن، فیلتر کردن یا آنالیز روی یک ستون خاص خیلی سریع‌تر انجام بشه

این مفهوم رو ساده‌سازی کردیم ('Date' بدون) اینجا ما با انتخاب زیرمجموعه‌ای از ستون‌ها

column_based_data = sales_df[['ProductID', 'Region', 'SalesAmount']]

در نهایت، داده‌های شبیه‌سازی شده مبتنی بر ستون رو چاپ می‌کنیم

print("Simulated Column-based Data:")

print(column_based_data)

Simulated Column-based Data:

	ProductID	Region	SalesAmount
0	1	North	500
1	2	South	300
2	1	North	700
3	2	South	450

توضیحات مثال کد پایتون

در این مثال، با انتخاب تنها ستون‌های مرتبط از داده‌های فروش اصلی، روشی شبیه به **ذخیره‌سازی مبتنی بر ستون** را شبیه‌سازی کرده‌ایم.

اگرچه این شبیه‌سازی ساده است، پایگاه‌های داده واقعی مبتنی بر ستون مانند **Apache Cassandra**، **Amazon Redshift** یا **Google BigQuery** امکانات پیشرفته‌تری در زمینه ذخیره‌سازی ستونی و بهینه‌سازی عملکرد ارائه می‌دهند. این پایگاه‌های داده برای اجرای کارآمد پرس‌وجوهای تحلیلی روی مجموعه‌داده‌های بزرگ طراحی شده‌اند و بنابراین برای کاربردهای داده‌کاوی و تحلیل داده بسیار مناسب هستند.

Section 3.4

3.4.1. Terminology of data cube computation

اصطلاحات رایج در مکعب‌های داده و OLAP

در زمینه مکعب‌های داده و OLAP، اصطلاحات خاصی به‌طور رایج برای توصیف محاسبات و عملیات انجام‌شده بر روی داده‌های چندبعدی استفاده می‌شوند. درک این اصطلاحات برای انجام مؤثر تحلیل داده و استخراج بینش‌های معنادار ضروری است.

اصطلاحات محاسبه در مکعب داده

برش (Slice): یک دید دو بعدی از مکعب داده که با ثابت کردن یک یا چند بعد در مقادیر خاص به دست می‌آید. این عمل به کاربران امکان می‌دهد بر یک زیرمجموعه از داده‌ها تمرکز کنند.

تاس‌زدن (Dice): ایجاد یک زیرمکعب با انتخاب مقادیر خاص از دو یا چند بعد. این عمل با مشخص کردن شرایط در ابعاد متعدد، دید تحلیل را محدودتر می‌کند.

فشردن (Roll-up / Drill-Up): تجمیع داده‌ها در سطح بالاتری از گرانولاریته با فرو ریختن یک یا چند بعد. این عمل به معنای حرکت از دید جزئی به دید خلاصه‌تر است.

جزئی‌سازی (Drill-down / Roll-Down): باز کردن داده‌های تجمیع‌شده به سطح پایین‌تری از گرانولاریته با گسترش یک یا چند بعد. این عمل به معنای حرکت از دید خلاصه به دید جزئی‌تر است.

چرخش (Pivot / Rotate): تغییر جهت‌گیری مکعب داده برای مشاهده آن از دیدگاهی دیگر. این عمل شامل جابه‌جایی سطرها و ستون‌های مکعب داده است.

مثالی از محاسبات مکعب داده در پایتون

بیایید یک مجموعه داده فروش را در نظر بگیریم و چند عملیات محاسباتی از مکعب داده را با استفاده از کتابخانه Pandas انجام دهیم:

In [18]: `import pandas as pd`

رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالی به 'pandas' این قسمت اول کده. ما کتابخانه #

```
# این دیکشنری حاوی داده‌های فروش نمونه ماست
# رو داره 'Date' و 'Region' در نظر بگیرید که ابعاد مختلفی مثل (Data Cube) این داده‌ها رو به عنوان یک مکعب داده
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# اصلی رو چاپ می‌کنیم تا ببینیم داده‌ها چه شکلی هستن
print("Original Sales Data:")
print(sales_df)

# رو انجام می‌ده (Slice)** این بخش، عملیات **برش
# برش، انتخاب داده‌ها بر اساس یک بعد خاصه
# اینجا، ما فقط داده‌های مربوط به ماه ژانویه (ماه 1) رو انتخاب می‌کنیم
january_sales = sales_df[sales_df['Date'].dt.month == 1]

# رو انجام می‌ده (Dice)** این بخش، عملیات **مکعب داده
# مکعب داده، انتخاب داده‌ها بر اساس ترکیبی از چند بعد
# باشن و هم مربوط به ماه ژانویه 'North' اینجا، ما داده‌هایی رو انتخاب می‌کنیم که هم مربوط به منطقه
north_january_sales = sales_df[(sales_df['Region'] == 'North') & (sales_df['Date'].dt.month == 1)]

# در نهایت، نتایج هر دو عملیات رو چاپ می‌کنیم
print("\nOLAP Slice Operation (January Sales):")
print(january_sales)

print("\nOLAP Dice Operation (North January Sales):")
print(north_january_sales)
```

Original Sales Data:

	Date	ProductID	Region	SalesAmount
0	2022-01-01	1	North	500
1	2022-01-02	2	South	300
2	2022-02-01	1	North	700
3	2022-02-02	2	South	450

OLAP Slice Operation (January Sales):

	Date	ProductID	Region	SalesAmount
0	2022-01-01	1	North	500
1	2022-01-02	2	South	300

OLAP Dice Operation (North January Sales):

	Date	ProductID	Region	SalesAmount
0	2022-01-01	1	North	500

توضیحات مثال کد پایتون

در این مثال، یک عملیات **برش (slice)** با انتخاب داده‌های فروش ماه ژانویه و یک عملیات **تاس‌زدن (dice)** با انتخاب داده‌های فروش منطقه شمال در همان ماه انجام شده است.

این اصطلاحات محاسبه در مکعب داده نشان می‌دهند که چگونه کاربران می‌توانند به‌طور مؤثر در داده‌های چندبعدی حرکت کرده و آن‌ها را تحلیل کنند. مثال ارائه‌شده، این عملیات‌ها را در یک سناریوی عملی با استفاده از **Pandas** نشان می‌دهد.

3.4.2. Data cube materialization: ideas

متریالیزه کردن مکعب داده

متریالیزه کردن مکعب داده (Data Cube Materialization) به معنای **پیش‌محاسبه** و ذخیره داده‌های تجمیع‌شده در یک مکعب داده برای تسریع عملکرد پرس‌وجو است. ایده اصلی این است که نتایج عملیات تجمیع از قبل محاسبه و ذخیره شوند تا نسخه‌ای خلاصه‌شده از داده اصلی ایجاد شود. مکعب‌های داده متریالیزه می‌توانند زمان پاسخ به پرس‌وجوها را به‌ویژه در موارد پرس‌وجوهای تکراری یا پیچیده تحلیلی به‌طور چشمگیری کاهش دهند.

رویکردهای متریالیزه کردن مکعب داده

متریالیزه کامل (Full Materialization): محاسبه و ذخیره تمام ترکیبات ممکن از مقادیر تجمیع‌شده در مکعب داده. این روش سریع‌ترین زمان پاسخ به پرس‌وجو را فراهم می‌کند، اما ممکن است به فضای ذخیره‌سازی بسیار زیادی نیاز داشته باشد.

متریالیزه جزئی (Partial Materialization): محاسبه و ذخیره انتخابی از تجمیع‌ها یا زیرمجموعه‌های مکعب داده بر اساس الگوهای پیش‌بینی‌شده پرس‌وجو. این روش تعادلی بین عملکرد پرس‌وجو و محدودیت فضای ذخیره‌سازی ایجاد می‌کند.

متریالیزه Top-N (Top-N Materialization): فقط N تجمیع یا ترکیب برتر متریالیزه می‌شوند، که در آن N نشان‌دهنده پرس‌وجوهای پرتکرار یا بینش‌های باارزش است. این روش برای موارد استفاده رایج بهینه‌سازی شده است.

متریالیزه افزایشی (Incremental Materialization): به‌روزرسانی افزایشی مکعب داده متریالیزه هنگامی که داده‌های جدیدی در دسترس قرار می‌گیرند. این امر تضمین می‌کند که مکعب متریالیزه همیشه با داده‌های پایه به‌روز باقی بماند.

مثالی از متریالیزه کردن مکعب داده در پایتون

اگرچه متریالیزه کردن واقعی معمولاً در پایگاه‌های داده تخصصی انجام می‌شود، اما می‌توانیم با استفاده از کتابخانه Pandas در پایتون، یک سناریوی متریالیزه جزئی را شبیه‌سازی کنیم:

In [19]: **import** pandas **as** pd

```
# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست.
# استفاده می‌شن OLAP در سیستم‌های ** (Partial Materialization) این داده‌ها برای نمایش مفهوم **ماده‌سازی جزئی
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو شبیه‌سازی می‌کنه ** (Partial Materialization) این بخش، **ماده‌سازی جزئی
# ماده‌سازی یعنی نتایج پرس‌وجوها یا محاسبات رو از قبل حساب کرد و ذخیره کرد، OLAP تو سیستم‌های
# ماده‌سازی جزئی به این معنیۀ که فقط بخشی از مکعب داده (مثلاً تجمیع‌های پرتکرار) ماده‌سازی بشه، نه همه ترکیب‌های ممکن
# از قبل محاسبه و ذخیره می‌کنیم 'Region' اینجا ما میانگین فروش رو بر اساس
materialized_cube = sales_df.groupby('Region')['SalesAmount'].mean().reset_index()

# در نهایت، مکعب ماده‌سازی شده رو چاپ می‌کنیم
print("Materialized Cube (Average Sales by Region):")
print(materialized_cube)
```

```
Materialized Cube (Average Sales by Region):
  Region  SalesAmount
0  North           600.0
1  South           375.0
```

توضیحات مثال کد پایتون

در این مثال، با محاسبه و ذخیره میانگین مقدار فروش برای هر منطقه، یک متریالیزه جزئی را شبیه‌سازی کرده‌ایم. در سناریوهای واقعی، متریالیزه کردن ممکن است شامل تجمیع‌های پیچیده‌تر و بهینه‌سازی‌های ذخیره‌سازی باشد.

این مثال مفهوم متریالیزه کردن مکعب داده را نشان می‌دهد و چگونگی تعادل بین عملکرد پرس‌وجو و فضای ذخیره‌سازی را توضیح می‌دهد.

معماری‌های سرور OLAP

معماری‌های سرور OLAP (پردازش تحلیلی آنی) برای پشتیبانی از تحلیل و پرس‌وجوی داده‌های چندبعدی طراحی شده‌اند. سه نوع اصلی معماری OLAP وجود دارد: **ROLAP** (OLAP نسبی)، **MOLAP** (OLAP چندبعدی) و **HOLAP** (OLAP ترکیبی). هر معماری ویژگی‌ها و موارد استفاده خاص خود را دارد.

انواع معماری‌های OLAP

1. ROLAP (Relational OLAP): سرورهای ROLAP داده‌ها را در پایگاه‌های داده نسبی (RDBMS) ذخیره می‌کنند و نتایج چندبعدی را به‌صورت پویا و با استفاده از پرس‌وجوهای SQL تولید می‌کنند. تجمیع‌ها و محاسبات در همان سیستم مدیریت پایگاه داده انجام می‌شوند. **کاربرد:** مناسب برای مجموعه‌داده‌های بزرگ که در آن‌ها تولید پیش‌فرض تمام ترکیبات در یک مکعب چندبعدی عملی نیست.

2. MOLAP (Multidimensional OLAP): سرورهای MOLAP داده‌ها را در قالب اختصاصی و چندبعدی ذخیره می‌کنند که برای پردازش تحلیلی بهینه‌سازی شده است. داده‌ها از قبل تجمیع شده و در ساختار مکعب ذخیره می‌شوند و این امر منجر به **سرعت بالای پاسخ به پرس‌وجو** می‌شود. **کاربرد:** برای سناریوهایی که عملکرد سریع پرس‌وجو حیاتی است و فضای ذخیره‌سازی محدودیت اصلی محسوب نمی‌شود.

3. HOLAP (Hybrid OLAP): سرورهای HOLAP ترکیبی از ROLAP و MOLAP هستند. تجمیع‌ها و خلاصه‌ها برای داده‌های پرتکرار در یک مکعب چندبعدی (MOLAP) ذخیره می‌شوند، در حالی که داده‌های جزئی در پایگاه داده نسبی (ROLAP) نگهداری می‌شوند. **کاربرد:** تعادلی بین **عملکرد پرس‌وجو و مصرف فضای ذخیره‌سازی** ایجاد می‌کند و از مزایای هر دو معماری بهره می‌برد.

مثالی از ROLAP، MOLAP و HOLAP در پایتون

در این مثال، از **Pandas** همراه با یک پایگاه داده نسبی (مانند SQLite) برای شبیه‌سازی **ROLAP** استفاده می‌کنیم، از آرایه‌های چندبعدی با **xarray** یا **NumPy** برای مدل‌سازی **MOLAP** استفاده می‌کنیم، و ترکیبی از هر دو رویکرد را برای شبیه‌سازی یک سیستم **HOLAP** به کار می‌بریم — جایی که داده‌های خلاصه‌شده در حافظه نگهداری می‌شوند و داده‌های جزئی از پایگاه داده واکنشی می‌شوند.

```
In [23]: import pandas as pd

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیه 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# استفاده می‌شن OLAP این داده‌ها برای شبیه‌سازی و مقایسه سه معماری ذخیره‌سازی و پردازش داده در سیستم‌های
# ROLAP (Relational OLAP)، MOLAP (Multidimensional OLAP)، و HOLAP (Hybrid OLAP).
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
```

```

sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو شبیه‌سازی می‌کنه **ROLAP (Relational OLAP)**، این بخش
# نگه‌داری می‌شن (Relational DB) داده‌ها در پایگاه‌های داده رابطه‌ای، ROLAP، تو
# برای تجزیه و تحلیل چندبعدی استفاده می‌شه SQL و از دستورات
# عمل می‌کنه SQL در `GROUP BY` پانداس استفاده می‌کنیم که مشابه `groupby()` برای شبیه‌سازی کوئری، از تابع
rolap_result = sales_df.groupby('Region')['SalesAmount'].sum().reset_index()
print("نتایج شبیه‌سازی ROLAP:")
print(rolap_result)
print("="*30)

# رو شبیه‌سازی می‌کنه **MOLAP (Multidimensional OLAP)**، این بخش
# ذخیره می‌شن (Multidimensional Cube) داده‌ها در قالب یک مکعب چندبعدی، MOLAP، تو
# این ساختار، پرس‌وجوهای تحلیلی رو خیلی سریع‌تر می‌کنه
# پانداس استفاده می‌کنیم که مکعب داده‌ها رو شبیه‌سازی می‌کنه `pivot_table` ما از تابع
molap_result = pd.pivot_table(
    sales_df,
    values='SalesAmount',
    index=['Date', 'ProductID'],
    columns=['Region'],
    aggfunc='sum'
)
print("نتایج شبیه‌سازی MOLAP:")
print(molap_result)
print("="*30)

# رو شبیه‌سازی می‌کنه **HOLAP (Hybrid OLAP)**، این بخش
# هست و سعی می‌کنه نقاط قوت هر دو رو ترکیب کنه MOLAP و ROLAP ترکیبی از HOLAP
# ما از یک پایوت تیبل ساده برای نمایش این معماری ترکیبی استفاده می‌کنیم
holap_result = pd.pivot_table(
    sales_df,
    values='SalesAmount',
    index=['Date', 'ProductID'],
    columns=['Region'],
    aggfunc='sum'
)
print("نتایج شبیه‌سازی HOLAP:")
print(holap_result)
print("="*30)

```

ROLAP نتایج شبیه‌سازی:

	Region	SalesAmount
0	North	1200
1	South	750

=====

MOLAP نتایج شبیه‌سازی:

Region	North	South
Date ProductID		
2022-01-01 1	500.0	NaN
2022-01-02 2	NaN	300.0
2022-02-01 1	700.0	NaN
2022-02-02 2	NaN	450.0

=====

HOLAP نتایج شبیه‌سازی:

Region	North	South
Date ProductID		
2022-01-01 1	500.0	NaN
2022-01-02 2	NaN	300.0
2022-02-01 1	700.0	NaN
2022-02-02 2	NaN	450.0

=====

توضیحات مثال کد پایتون

در این مثال، **ROLAP** را با استفاده از پرس‌وجوهای SQL و کتابخانه **MOLAP**، Pandas، و PyDataCube استفاده از کتابخانه PyDataCube و **HOLAP** را با ترکیب Pandas و PyDataCube در یک گردش کار تحلیلی شبیه‌سازی کرده‌ایم. هر رویکرد، یکی از معماری‌های سرور OLAP را نشان می‌دهد و تفاوت‌های آن‌ها در تعادل بین عملکرد پرس‌وجو و کارایی ذخیره‌سازی را به خوبی برجسته می‌کند.

3.4.4. General strategies for data cube computation

راهبردهای محاسبه مکعب داده

محاسبه مکعب داده شامل راهبردهایی برای تولید و مدیریت کارآمد مکعب‌های داده چندبعدی است که برای پردازش تحلیلی آنی (OLAP) و داده‌کاوی ضروری هستند. در ادامه برخی از راهبردهای کلی برای محاسبه مکعب داده ارائه شده است:

1. **تجمیع و پیش‌محاسبه (Aggregation and Precomputation):** محاسبه و ذخیره از پیش مقادیر تجمیع‌شده در مکعب داده به منظور تسریع عملکرد پرس‌وجو.

مثال در پایتون: استفاده از Pandas برای محاسبه و ذخیره مقادیر تجمیع‌شده در مرحله اولیه و قبل از اجرای پرس‌وجوها.

In [24]:

```
import pandas as pd

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالی‌ه 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# استفاده می‌شن **Precomputed Aggregation** این داده‌ها برای نمایش مفهوم **پیش‌محاسبه و ذخیره‌سازی داده‌های تجمیع‌شده
# هست (OLAP مانند) این کار یکی از روش‌های بهینه‌سازی عملکرد در سیستم‌های تحلیلی
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو انجام می‌ده **Region** این بخش، **تجمیع داده‌ها بر اساس بعد
# یعنی محاسبه مقادیر خلاصه‌ای مثل جمع، میانگین یا شمارش (Aggregation) تجمیع
# اینجا ما جمع کل فروش رو برای هر منطقه از قبل محاسبه می‌کنیم
# این کار به جای اینکه هر بار کوئری اجرا بشه داده‌ها رو جمع کنه، عملکرد رو بهبود می‌ده
# هست OLAP در سیستم‌های (Materialization) این روش شبیه به ماده‌سازی
aggregated_data = sales_df.groupby('Region')['SalesAmount'].sum().reset_index()

# در نهایت، داده‌های پیش‌محاسبه شده رو چاپ می‌کنیم
print("Precomputed Aggregated Data:")
print(aggregated_data)
```

Precomputed Aggregated Data:

	Region	SalesAmount
0	North	1200
1	South	750

2. **ایندکس‌گذاری و پارتیشن‌بندی (Indexing and Partitioning):** استفاده از ایندکس‌گذاری و پارتیشن‌بندی داده‌ها برای بهینه‌سازی محاسبه و ذخیره‌سازی مکعب داده. این روش‌ها دسترسی سریع‌تر به زیرمجموعه‌های داده و کاهش بار محاسباتی را فراهم می‌کنند.

مثال در پایتون: استفاده از تکنیک‌های ایندکس‌گذاری کارآمد و پارتیشن‌بندی در Pandas یا کتابخانه‌های دیگر مانند Dask یا PyArrow.

In [25]:

```
import pandas as pd

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالی‌ه 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# در سیستم‌های مدیریت داده و تحلیلی استفاده می‌شن **Indexing** این داده‌ها برای نمایش مفهوم **ایندکس‌گذاری
# ایندکس‌گذاری یک تکنیک بهینه‌سازی هست که سرعت دسترسی به داده‌ها رو، به‌ویژه در عملیات فیلتر و جستجو، به‌شدت افزایش می‌ده
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو انجام می‌ده **Region** این بخش، **ایندکس‌گذاری روی ستون
# به عنوان ایندکس دیتافریم در نظر گرفته می‌شه 'Region' ستون، `set_index` با استفاده از
# خیلی سریع‌تر انجام بشه ('North' مثل) این کار باعث می‌شه جستجو و فیلتر داده‌های مربوط به یک منطقه خاص
# این روش مشابه ایندکس‌های موجود در پایگاه‌های داده رابطه‌ای هست و برای بهبود عملکرد در عملیات تکراری بسیار مفیده
sales_df.set_index('Region', inplace=True)

# در نهایت، داده‌های ایندکس‌شده رو چاپ می‌کنیم تا ساختار جدید رو ببینیم
print("Indexed Sales Data:")
print(sales_df)
```

Indexed Sales Data:			
	Date	ProductID	SalesAmount
Region			
North	2022-01-01	1	500
South	2022-01-02	2	300
North	2022-02-01	1	700
South	2022-02-02	2	450

3. پردازش موازی (Parallel Processing): توزیع وظایف محاسبه مکعب داده بین چندین پردازنده یا گره به منظور اجرای موازی. این روش زمان محاسبه را به‌طور چشمگیری کاهش می‌دهد، به‌ویژه برای مجموعه‌داده‌های بزرگ.

مثال در پایتون: استفاده از کتابخانه‌های پردازش موازی مانند **Dask** یا ماژول **multiprocessing** در پایتون برای موازی‌سازی عملیات تجمیع و محاسبه مکعب.

```
In [ ]: pip install "dask[dataframe]"
```

```
In [ ]: pip install --upgrade pyarrow
```

```
In [1]: import pandas as pd
import dask.dataframe as dd

# رو وارد می‌کنیم 'dask' و 'pandas' این قسمت اول کده. ما کتابخانه‌های
# 'pandas' و مقیاس‌پذیر داده‌های بزرگ استفاده می‌شه 'dask' برای کار با داده‌ها و

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# استفاده می‌شن Dask با استفاده از کتابخانه *(Parallel Processing) این داده‌ها برای نمایش مفهوم **پردازش موازی
# اجازه می‌ده تا عملیات روی دیتافریم‌ها به صورت موازی و روی چندین هسته انجام بشه Dask
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو انجام می‌ده Dask به دیتافریم pandas این بخش، تبدیل دیتافریم
# تقسیم می‌کنه و امکان پردازش موازی رو فراهم می‌کنه (partitions) داده‌ها رو به بخش‌های کوچک‌تر Dask
# استفاده کنیم CPU این ویژگی به‌ویژه زمانی مفیده که با داده‌های خیلی بزرگ کار می‌کنیم و می‌خوایم از تمام قدرت
dask_sales_df = dd.from_pandas(sales_df, npartitions=2)

# روی داده‌ها انجام می‌دیم (sum و groupby) اینجا یک عملیات تجمیع
# نتیجه نهایی تولید می‌شه `.compute()` این عملیات به صورت موازی روی پارتیشن‌ها اجرا می‌شه و با فراخوانی
# این روش می‌تونه عملکرد رو در داده‌های حجیم به‌طور چشمگیری بهبود بده
result = dask_sales_df.groupby('Region')['SalesAmount'].sum().compute()

# در نهایت، نتیجه عملیات موازی رو چاپ می‌کنیم
print("Result of Parallel Processing:")
print(result)
```

Result of Parallel Processing:
Region
North 1200
South 750
Name: SalesAmount, dtype: int64

توضیحات مثال کد پایتون

این راهبردها نشان‌دهنده رویکردهای عملی برای بهبود کارایی محاسبه مکعب داده هستند. انتخاب راهبرد مناسب به نیازهای خاص تحلیل بستگی دارد، از جمله حجم داده، پیچیدگی پرس‌وجوها و منابع محاسباتی در دسترس. ترکیب این روش‌ها می‌تواند عملکرد تحلیل‌های OLAP و داده‌کاوی را به‌طور چشمگیری ارتقا دهد.

Section 3.5

3.5.1. Multiway array aggregation for full cube computation

جمع‌بندی آرایه چندطرفه برای محاسبه کامل مکعب

روش **جمع‌بندی آرایه چندطرفه** (Multiway Array Aggregation) شامل تجميع داده‌ها در یک آرایه چندبعدی است که امکان محاسبه کارآمد و جامع کل مکعب داده را فراهم می‌کند. این رویکرد به‌ویژه در کار با مجموعه داده‌های بزرگ و پرس‌وجوهای تحلیلی پیچیده بسیار مفید است. آرایه‌های چندطرفه (که به آن‌ها تنسور نیز گفته می‌شود)، نمایشی فشرده و ساختاریافته از داده‌های چندبعدی ارائه می‌دهند و تجميع آن‌ها امکان محاسبه همزمان ابعاد مختلف مکعب را ساده و کارآمد می‌کند.

مفاهیم کلیدی جمع‌بندی آرایه چندطرفه

1. نمایش تنسوری (Tensor Representation): تنسور یک آرایه چندبعدی است که می‌تواند داده‌ها را در بیش از دو بعد نمایش دهد. در زمینه مکعب‌های داده، یک تنسور سه‌بعدی می‌تواند داده‌ها را در ابعادی مانند زمان، محصول و منطقه نمایش دهد.

2. عملیات تجميع (Aggregation Operations): عملیات تجميع روی تنسورها شامل فرو ریختن یا ترکیب مقادیر در امتداد یک یا چند بعد است. در مکعب داده، این عملیات ممکن است شامل **جمع**، **میانگین‌گیری** یا سایر توابع تجميعی باشد.

3. محاسبه کارآمد (Efficient Computation): جمع‌بندی آرایه چندطرفه راهی کارآمد برای انجام محاسبه کامل مکعب فراهم می‌کند، بدون نیاز به ایجاد صریح و ذخیره یک مکعب چندبعدی. تجميع‌ها به‌صورت پویا انجام می‌شوند و این امر **نیاز به فضای ذخیره‌سازی** را به حداقل می‌رساند.

مثالی از جمع‌بندی آرایه چندطرفه در پایتون

بیایید از کتابخانه **NumPy** برای شبیه‌سازی یک تنسور سه‌بعدی استفاده کنیم که داده‌های فروش را نشان می‌دهد و سپس عملیات تجميع را روی آن انجام دهیم:

In [2]: **import** numpy **as** np

رو وارد می‌کنیم که برای کار با آرایه‌های چندبعدی و محاسبات عددی عالی 'numpy' این قسمت اول کده. ما کتابخانه

ایجاد می‌کنه (Data Cube) این بخش، یک ****تنسور سه‌بعدی**** رو برای شبیه‌سازی داده‌های فروش در قالب مکعب داده.
ابعاد تنسور به ترتیب زمان (مثلاً 3 ماه)، محصول (مثلاً 4 محصول)، و منطقه (مثلاً 2 منطقه) هستن
و تحلیل داده‌های تجاری رو شبیه‌سازی می‌کنه OLAP این ساختار نمایش چندبعدی داده‌ها در سیستم‌های
sales_tensor = np.random.randint(100, 1000, size=(3, 4, 2))

تنسور شبیه‌سازی شده رو چاپ می‌کنیم تا ساختار داده‌های چندبعدی رو ببینیم
print("Simulated Sales Tensor:")
print(sales_tensor)

روی تمام ابعاد تنسور انجام می‌ده ****Aggregation**** این بخش، عملیات ****تجميع****
جمع کل فروش روی هر سه بعد (زمان، محصول، منطقه) محاسبه می‌شه، **axis=(0, 1, 2)** و مشخص کردن **np.sum** با استفاده از
این عملیات معادل به دست آوردن "کل فروش" در یک سیستم تحلیلی هست
در عمل، این نوع تجميع‌ها برای گزارش‌های خلاصه و داشبوردها کاربرد فراوانی دارن
total_sales = np.sum(sales_tensor, axis=(0, 1, 2))

نتیجه نهایی، یعنی کل فروش تجميع‌شده، رو چاپ می‌کنیم
print("\nTotal Sales (Aggregated):", total_sales)

Simulated Sales Tensor:
[[[605 148]
[974 902]
[265 909]
[932 371]]

[[811 394]
[930 683]
[812 204]
[689 860]]

[[233 758]
[545 319]
[867 588]
[425 563]]]

Total Sales (Aggregated): 14787



توضیحات مثال کد پایتون

این مثال مفهوم **جمع‌بندی آرایه چندطرفه** را نشان می‌دهد و چگونگی ارائه یک روش انعطاف‌پذیر و کارآمد برای انجام **محاسبه کامل مکعب داده** را توضیح می‌دهد. تأکید می‌شود که اگرچه این مثال از یک تنسور کوچک شبیه‌سازی‌شده استفاده می‌کند، اما این روش قابل مقیاس‌سازی به مجموعه‌داده‌های بزرگ‌تر و ابعاد پیچیده‌تر است.

3.5.2. BUC: computing iceberg cubes from the apex cuboid downward



روش BUC (محاسبه از بالا به پایین)

BUC (محاسبه از بالا به پایین) یک تکنیک برای محاسبه کارآمد **مکعب‌های آبشاری** (Iceberg Cubes) از جوهره بالایی (Apex Cuboid) – که خلاصه‌ترین سطح را نشان می‌دهد – به سمت پایین است. این روش تنها تجمیع‌هایی را محاسبه می‌کند که شرط از پیش تعیین‌شده‌ای را برآورده کنند، که به آن **شرط آبشاری** (Iceberg Condition) گفته می‌شود. با شروع از خلاصه‌ترین سطح و حرکت تدریجی به سمت سطوح جزئی‌تر، BUC پیچیدگی محاسباتی را کاهش داده و فقط بر تجمیع‌های معنادار و مرتبط تمرکز می‌کند.



مفاهیم کلیدی BUC

1. شرط آبشاری (Iceberg Condition): BUC از یک شرط آبشاری برای تعیین اینکه آیا یک تجمیع به اندازه کافی معنادار است یا نه، استفاده می‌کند. این شرط معمولاً شامل آستانه‌ای تعریف‌شده توسط کاربر است، مانند **حداقل پشتیبانی** در زمینه کاوش قواعد ارتباطی.

2. جوهره بالایی (Apex Cuboid): جوهره بالایی، سطح خلاصه‌ترین تجمیع در مکعب داده است که نمایی بسیار کلی و فشرده از داده‌ها را ارائه می‌دهد.

3. محاسبه از بالا به پایین (Bottom-Up Computation): BUC از جوهره بالایی شروع می‌کند و به تدریج تجمیع‌ها را به سمت سطوح پایین‌تر محاسبه می‌کند و از انجام محاسبات غیرضروری برای ترکیباتی که شرط آبشاری را برآورده نمی‌کنند، اجتناب می‌کند.



مثالی از BUC در پایتون

بیا یید یک مجموعه داده کوچک را شبیه‌سازی کنیم و از الگوریتم BUC برای محاسبه مکعب‌های آبخاری بر اساس یک آستانه مشخص استفاده کنیم:

```
In [4]: import pandas as pd
import numpy as np

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیه 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# استفاده می‌شن (Iceberg Cube) و ساخت مکعب‌های یخچالی **BUC (Bottom-Up Computation)** این داده‌ها برای اجرای الگوریتم
# مکعب یخچالی فقط تجمیع‌هایی رو نگه می‌داره که یک شرط آستانه‌ای (مثل مجموع فروش بیشتر از ۱۰۰۰) رو برآورده کنن
# این روش از ذخیره‌سازی تجمیع‌های بی‌معنی جلوگیری می‌کنه و فضای ذخیره‌سازی و زمان پردازش رو بهینه می‌کنه
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو شامل می‌شه **BUC (Bottom-Up Computation)** این بخش، پیاده‌سازی اصلاح‌شده و دقیق‌تر الگوریتم
# لاف نسخه قبلی که از بالا به پایین حرکت می‌کرد، این نسخه با رویکردی ساختاریافته‌تر، از جزئیات شروع نمی‌کنه بلکه به‌صورت بازگشتی و با مدیریت صحیح ابعاد، تجمیع‌ها رو بررسی می‌کنه
# هدف اینه که تمام ترکیب‌های ممکن از ابعاد رو به‌صورت هوشمند و بدون تکرار اضافی پیمایش کنیم
# (pruning) شرط "یخچالی" در هر سطح چک می‌شه و اگر مقدار تجمیع از آستانه کمتر باشه، شاخه قطع می‌شه
def buc(data, dimensions, measures, threshold):
    """
    رو برای ساخت مکعب یخچالی پیاده‌سازی می‌کنه BUC این تابع یک نسخه ساده‌شده از الگوریتم
    ورودی‌ها:
        data (pd.DataFrame): داده‌های خام شامل داده‌های خام.
        dimensions (list): لیستی از نام ستون‌هایی که باید روی اون‌ها تجمیع انجام بشه
        measures (str): نام ستونی که قراره مقدارش جمع بشه (مثلاً 'SalesAmount').
        threshold (float): حداقل مقداری که یک تجمیع باید داشته باشه تا ذخیره بشه

    خروجی:
        list: لیستی از دیکشنری‌ها که هر کدوم نماینده یک تجمیع معتبر هست
    """
    result = []

    def helper(df, current_dimensions, current_levels):
        # محاسبه مجموع سنجه برای داده‌های فعلی
        current_sum = df[measures].sum()

        # بررسی شرط یخچالی: آیا این تجمیع ارزش ذخیره شدن رو داره؟
        if current_sum >= threshold:
            # ساخت یک سطر خروجی که شامل مقادیر ابعاد و مقدار تجمیع هست
            agg_row = dict(zip(dimensions, current_levels))
            agg_row['TotalSales'] = current_sum
            result.append(agg_row)

            # اگر هنوز ابعادی برای پردازش باقی مونده، به سطح بعدی حرکت می‌کنیم
            if current_dimensions:
                first_dimension = current_dimensions[0]
                remaining_dimensions = current_dimensions[1:]

                # برای هر مقدار منحصریه‌فرد در بعد جاری، زیرمجموعه مربوطه رو جدا می‌کنیم
                for value in df[first_dimension].unique():
                    next_df = df[df[first_dimension] == value]
                    next_levels = current_levels + [value]
                    # فراخوانی بازگشتی برای ادامه تجزیه و تحلیل
                    helper(next_df, remaining_dimensions, next_levels)

    # شروع الگوریتم از سطح کلی (بدون تفکیک اولیه)
    # در اولین فراخوانی، هیچ بعدی اعمال نشده و همه داده‌ها در نظر گرفته می‌شن
    helper(sales_df, dimensions, [])

    return result

# با مشخص کردن ابعاد، سنجه و آستانه BUC اجرای الگوریتم
# فقط تجمیع‌هایی که مجموع فروششون حداقل ۱۰۰۰ باشه، در نتیجه ظاهر می‌شن
buc_result = buc(sales_df, dimensions=['Date', 'ProductID', 'Region'], measures='SalesAmount', threshold=1000)

# در نهایت، نتایج تجمیع‌هایی که شرط یخچالی رو داشتن رو چاپ می‌کنیم
print("BUC Result:")
print(buc_result)
```

BUC Result:
[{'TotalSales': 1950}]

توضیحات مثال کد پایتون

در این مثال، تابع **buc** یک مجموعه داده، ابعاد، معیارها و یک آستانه را به عنوان ورودی گرفته و به‌صورت بازگشتی مکعب‌های آبخاری را بر اساس الگوریتم BUC محاسبه می‌کند. نتیجه تنها شامل تجمیع‌هایی است که شرط آستانه مشخص‌شده را برآورده می‌کنند.

این مثال نشان می‌دهد که چگونه الگوریتم BUC به‌طور کارآمد فضای جست‌وجو را کاهش می‌دهد و تنها بر تجمیع‌های مرتبط تمرکز می‌کند. برای روشن‌سازی، از یک مجموعه داده کوچک استفاده شده است.

3.5.3. Precomputing shell fragments for fast high-dimensional OLAP

پیش‌محاسبه تکه‌های پوسته

پیش‌محاسبه تکه‌های پوسته (Shell Fragments) یک راهبرد برای اجرای سریع OLAP (پردازش تحلیلی آنی) در محیط‌های با ابعاد بالا است. در سناریوهای OLAP با ابعاد فراوان، انجام تجمیع‌ها به‌صورت پویا از نظر محاسباتی بسیار پرهزینه می‌شود. با پیش‌محاسبه تکه‌های پوسته، تجمیع‌های مربوط به زیرمجموعه‌هایی از ابعاد محاسبه و ذخیره می‌شوند و ساختارهای پوسته‌ای ایجاد می‌شوند که می‌توانند به‌صورت کارآمدی ترکیب شوند تا به پرس‌وجوهای پیچیده پاسخ دهند.

مفاهیم کلیدی پیش‌محاسبه تکه‌های پوسته

- ساختار پوسته (Shell Structure):** پوسته، نمایانگر یک زیرمجموعه از ابعاد در مکعب داده است. در OLAP‌های با ابعاد بالا، پیش‌محاسبه و ذخیره تمام ترکیبات ممکن ابعاد عملی نیست. به جای آن، تکه‌های پوسته برای زیرمجموعه‌هایی از ابعاد محاسبه می‌شوند.
- پیش‌محاسبه تکه‌ها (Fragment Precomputation):** تجمیع‌ها برای هر تکه پوسته از پیش محاسبه شده و اطلاعات خلاصه‌شده برای ترکیبات خاصی از ابعاد ذخیره می‌شوند.
- پردازش کارآمد پرس‌وجو (Efficient Query Processing):** در حین اجرای پرس‌وجو، تکه‌های پوسته به‌صورت کارآمدی ترکیب می‌شوند تا به پرس‌وجوهای پیچیده OLAP پاسخ داده شود، بدون نیاز به تجمیع روی کل فضای با ابعاد بالا.

مثالی از پیش‌محاسبه تکه‌های پوسته در پایتون

بیایید یک مجموعه داده را شبیه‌سازی کنیم و نحوه پیش‌محاسبه تکه‌های پوسته را در یک سناریوی OLAP با ابعاد بالا نشان دهیم:

```
In [5]: import pandas as pd
import itertools

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیه 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه با ابعاد بالاست.
# هستن 'Category' و 'Region'، 'ProductID'، 'Date' این داده‌ها شامل ابعاد مختلفی مثل
# (Dimensionality Curse) مشهور به "فاجعه ابعاد" یا با افزایش تعداد ابعاد، تعداد ترکیبات ممکن برای تجمیع به‌سرعت رشد می‌کنه، OLAP در سیستم‌های
# استفاده می‌شه ** (Shell Fragments) برای مقابله با این چالش، از تکنیک‌هایی مثل **قطعه‌های پوسته
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'Category': ['Electronics', 'Clothing', 'Electronics', 'Clothing'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روش انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو پیش‌محاسبه می‌کنه ** (Shell Fragments) این بخش، **قطعه‌های پوسته
# از قبل محاسبه و ذخیره می‌شن ('ProductID' و 'Date' مثلاً تمام ترکیب‌های ممکن) هستن که در اون تجمیع‌های زیرمجموعه‌ای از ابعاد OLAP قطعه‌های پوسته یک تکنیک بهینه‌سازی در
# این کار به جای ساخت تمام ترکیبات ممکن (که با ابعاد بالا غیرعملی می‌شه)، فقط زیرمجموعه‌های کوچک و پرتکرار رو ماده‌سازی می‌کنه
```

```
# بعداً می‌تونیم از این قطعات برای ساخت سریع تجمیع‌های مرکب استفاده کنیم
def precompute_shell_fragments(data, dimensions, measures):
    fragments = {}

    # تولید تمام ترکیب‌های ممکن از ابعاد (از اندازه ۱ تا کل ابعاد)
    # ساخته می‌شن ('Date', 'ProductID') و ('ProductID',) و ('Date',) و ترکیب‌های ['Date', 'ProductID'] مثلاً برای
    all_combinations = [list(combination) for r in range(1, len(dimensions) + 1) for combination in itertools.combinations(dimensions, r)]

    for combination in all_combinations:
        fragment_key = tuple(combination)
        # تجمیع داده‌ها بر اساس ترکیب فعلی ابعاد
        fragment_data = data.groupby(combination)[measures].sum().reset_index()
        fragments[fragment_key] = fragment_data

    return fragments

# اجرای تابع برای ابعاد ['Date', 'ProductID']
# اینجا ما فقط یک زیرمجموعه کوچک از ابعاد رو برای نمایش قطعه‌های پوسته انتخاب کردیم
fragments = precompute_shell_fragments(sales_df, dimensions=['Date', 'ProductID'], measures='SalesAmount')

# در نهایت، یکی از قطعات پیش‌محاسبه شده رو چاپ می‌کنیم تا ساختارش رو ببینیم
print("Precomputed Shell Fragment for ['Date', 'ProductID']:")
print(fragments[('Date', 'ProductID')])
```

```
Precomputed Shell Fragment for ['Date', 'ProductID']:
   Date  ProductID  SalesAmount
0 2022-01-01         1          500
1 2022-01-02         2          300
2 2022-02-01         1          700
3 2022-02-02         2          450
```

توضیحات مثال کد پایتون

در این مثال، تابع `precompute_shell_fragments` تمام ترکیبات ممکن ابعاد را تولید کرده و تجمیع‌ها را برای هر تکه پوسته از پیش محاسبه می‌کند. نتیجه یک دیکشنری است که کلیدهای آن، تاپل‌هایی از ترکیبات ابعاد و مقادیر آن‌ها دیتافریم‌هایی شامل تجمیع‌های پیش‌محاسبه‌شده هستند.

این مثال مفهوم **پیش‌محاسبه تکه‌های پوسته** را نشان می‌دهد و چگونه به تسریع پردازش OLAP در فضاهای با ابعاد بالا کمک می‌کند.

3.5.4. Efficient processing of OLAP queries using cuboids

پردازش کارآمد پرس‌وجوهای OLAP با استفاده از جوهره‌ها

پردازش کارآمد پرس‌وجوهای OLAP (پردازش تحلیلی آنی) با استفاده از **جوهره‌ها** (Cuboids)، شامل ایجاد و ذخیره استراتژیک ساختارهای داده خلاصه‌شده است. جوهره‌ها ترکیبات مختلف ابعاد و سلسله مراتب در یک مکعب داده را نشان می‌دهند و با پیش‌محاسبه و ذخیره تجمیع‌ها در سطوح مختلف گرانولاریته، امکان پاسخ‌گویی سریع‌تر به پرس‌وجوها را فراهم می‌کنند. با سازماندهی داده‌ها در قالب جوهره‌ها، سیستم‌های OLAP می‌توانند به‌صورت کارآمد از سطوح مختلف جزئیات عبور کنند و نیاز به تجمیع‌های پویا را به حداقل برسانند.

مفاهیم کلیدی پردازش کارآمد با استفاده از جوهره‌ها

1. تعریف جوهره (Cuboid Definition): جوهره زیرمجموعه‌ای از یک مکعب داده است که ترکیب خاصی از ابعاد و سلسله مراتب را شامل می‌شود و نمایی خلاصه‌شده از داده‌ها را ارائه می‌دهد.

2. پیش‌محاسبه تجمیع‌ها (Precomputation of Aggregations): تجمیع‌ها در سطوح مختلف گرانولاریته درون هر جوهره از پیش محاسبه و ذخیره می‌شوند. این پیش‌محاسبه، پردازش پرس‌وجوها را به‌طور چشمگیری تسریع می‌کند.

3. حرکت در سلسله مراتب (Hierarchy Navigation): جوهره‌ها امکان حرکت کارآمد در سطوح مختلف سلسله مراتب و ابعاد

را فراهم می‌کنند و به کاربران اجازه می‌دهند با عملیات جزئی‌سازی (Drill-down) یا فشردن (Roll-up) داده‌ها را در سطوح

مختلف جزئیات کاوش کنند.

مثالی از پردازش کارآمد با استفاده از جوهره‌ها در پایتون

بیااید یک مجموعه داده را شبیه‌سازی کنیم و مفهوم جوهره‌ها را برای پردازش کارآمد پرس‌وجوهای OLAP نمایش دهیم:

In [9]:

```
import pandas as pd
import itertools

# رو وارد می‌کنیم که برای کار با داده‌ها تو قالب جدول عالیۀ 'pandas' این قسمت اول کده. ما کتابخونه

# این دیکشنری حاوی داده‌های فروش نمونه ماست
# هستن ('SalesAmount') و یک سنجۀ ('ProductID', 'Region', 'Category') مثلاً این داده‌ها شامل چندین بعد
# و پردازش پرس‌وجوها با تجمیع‌های از قبل محاسبه شده استفاده می‌شه (**Cuboid) مثل **کیوبیوید OLAP این ساختار برای شبیه‌سازی مفاهیم
data = {
    'Date': ['2022-01-01', '2022-01-02', '2022-02-01', '2022-02-02'],
    'ProductID': [1, 2, 1, 2],
    'Region': ['North', 'South', 'North', 'South'],
    'Category': ['Electronics', 'Clothing', 'Electronics', 'Clothing'],
    'SalesAmount': [500, 300, 700, 450]
}

sales_df = pd.DataFrame(data)
# تبدیل می‌کنه (datetime) رو به نوع داده زمانی 'Date' این خط، ستون
# تا بتونیم عملیات مربوط به زمان رو روشن انجام بدیم
sales_df['Date'] = pd.to_datetime(sales_df['Date'])

# رو پیاده‌سازی می‌کنه (**Cuboid) با استفاده از کیوبیوید OLAP این بخش، **مدیریت پرس‌وجوهای
# یک "کیوبیوید" به معنای یک سطح از تجمیع در مکعب چندبعدی هست، OLAP در سیستم‌های
# هر ترکیب از ابعاد، یک کیوبیوید مجزا تشکیل می‌ده.
# به جای محاسبه پویا هر پرس‌وجو، کیوبیویدها از قبل (یا به صورت پویا اما ذخیره‌شده) تولید می‌شن تا پاسخ‌دهی سریع‌تر بشه
# معروفه OLAP در (Materialization) "این روش به "ماده‌سازی"
def process_olap_queries_with_cuboids(data, dimensions, measures):
    # ایجاد کیوبیوید پایه (بدون بعد) – معادل کل فروش
    base_cuboid = pd.DataFrame({measures: [data[measures].sum()]})

    # دیکشنری برای ذخیره تمام کیوبیویدهای تولید شده
    cuboids = {}: base_cuboid

    # برای هر ترکیب ممکن از ابعاد، یک کیوبیوید ایجاد می‌کنیم
    for r in range(1, len(dimensions) + 1):
        for combination in itertools.combinations(dimensions, r):
            # تجمیع داده‌ها بر اساس ترکیب فعلی ابعاد
            cuboid_data = data.groupby(list(combination))[measures].sum().reset_index()
            cuboids[combination] = cuboid_data

    # 'South' مجموع فروش بر اساس دسته‌بندی در منطقه OLAP مثال از یک پرس‌وجوی
    # رو فیلتر می‌کنیم 'South' استفاده می‌کنیم و فقط ردیف‌های مربوط به ('Region', 'Category') اینجا از کیوبیوید
    query_result = cuboids[('Region', 'Category')].loc[
        cuboids[('Region', 'Category')]['Region'] == 'South'
    ]

    return query_result

# اجرای تابع با مشخص کردن تمام ابعاد و سنجۀ مورد نظر
result = process_olap_queries_with_cuboids(
    sales_df,
    dimensions=['Date', 'ProductID', 'Region', 'Category'],
    measures='SalesAmount'
)

# رو چاپ می‌کنیم OLAP در نهایت، نتیجۀ پرس‌وجوی
print("OLAP Query Result:")
print(result)
```

```
OLAP Query Result:
  Region Category SalesAmount
1  South  Clothing           750
```

توضیحات مثال کد پایتون

در این مثال، تابع `process_olap_queries_with_cuboids` جوهره‌های مربوط به ترکیبات مختلف ابعاد را ایجاد کرده و

تجمیع‌ها را از پیش محاسبه می‌کند. سپس یک پرس‌وجوی OLAP انجام می‌شود تا کل فروش در هر دسته‌بندی در منطقه

«جنوب» با استفاده از جوهره مربوط به ('Region', 'Category') استخراج شود.

جوهره‌ها با پیش‌محاسبه تجمیع‌ها در سطوح مختلف گرانولاریته، پردازش کارآمد پرس‌وجوهای OLAP را فراهم می‌کنند و زمان پاسخ به پرس‌وجو را به‌طور چشمگیری کاهش می‌دهند.

The End of Chapter 3

In []: